STQA

Software Testing & Quality Assurance

Complete Exam Preparation Guide

All 5 Modules • Top Questions • Full Answers • Exam Tips

Format: Attempt ANY THREE of SIX | 5 Questions × 15 Marks = 75 Total

MODULE 1 — Software Quality Management

STQA Exam Prep | Top Questions with Full Answers | Based on Past Paper Analysis

Q1. What is Quality? Core Components of Quality

★ **Appeared in BOTH past papers — 100% guaranteed**

Definition of Quality

Quality is the degree to which a product or service meets stated or implied customer requirements and expectations. In software, quality is both a fact (measurable attributes like defect count, performance) and a perception (subjective user satisfaction).

Core Components of Quality

Component	Description	Example
Functionality	Does the software do what it should?	Login feature works correctly
Reliability	Software performs consistently without failure	99.9% uptime
Usability	Ease of use by the end user	Intuitive UI, minimal training needed
Efficiency	Optimal use of resources	Fast response time, low memory use
Maintainability	Ease of modifying the software	Clean code, good documentation
Portability	Works across platforms/environments	Runs on Windows, Mac, Linux

> ✅ **EXAM TIP: Connect each component to a real example for full marks. The examiner loves application-level answers.**

Q2. Total Quality Management (TQM) — Principles & Quality Productivity Relationship

★ **Asked in both past papers — core topic**

Definition

TQM is a management philosophy focused on continuous improvement of all organizational processes, products, and services, with every employee involved in achieving quality.

Key TQM Principles

- Customer Focus — All activities are aimed at satisfying the customer
- Total Employee Involvement — Every employee participates in quality improvement
- Process-Centred Approach — Define and improve all work processes
- Integrated System — All departments work together toward common goals
- Strategic & Systematic Approach — Quality is part of the overall organizational strategy
- Continuous Improvement (Kaizen) — Constantly seeking better methods
- Fact-Based Decision Making — Decisions based on data and analysis

- Communications — Open and clear communication at all levels

Quality vs Productivity Relationship

Higher quality leads to higher productivity. When quality is improved: fewer defects → less rework → faster delivery → lower cost → higher productivity. This relationship is captured by the formula: Productivity = Output / Input. Improving quality reduces wasted input (rework, failures), increasing productivity.

Q3. Quality Management System (QMS) — Structure & Key Pillars

★ High chance — appeared in Paper 1

Definition of QMS

A Quality Management System is a formalized system that documents processes, procedures, and responsibilities to achieve quality policies and objectives. ISO 9001 is the most common QMS standard.

Key Pillars of QMS

- Quality Policy — Senior management's formal commitment to quality
- Quality Planning — Setting quality objectives and identifying processes needed to achieve them
- Quality Control — Monitoring and measuring activities to verify requirements are met
- Quality Assurance — Systematic activities providing confidence that quality requirements are fulfilled
- Quality Improvement — Ongoing actions to enhance quality performance

QMS Structure

Level	Document	Purpose
Level 1	Quality Manual	Overall quality policy and objectives
Level 2	Procedures	How activities are performed (who, what, when)
Level 3	Work Instructions	Detailed step-by-step task descriptions
Level 4	Records/Forms	Evidence of quality activities performed

Q4. Continuous Improvement Cycle (PDCA) & Benchmarking

★ High chance — asked in both papers

PDCA Cycle (Deming Cycle)

Phase	Full Form	Activities
P — Plan	Plan	Identify the problem, set objectives, plan changes
D — Do	Do	Implement the plan on a small scale
C — Check	Check	Measure and analyze results against expected
A — Act	Act	If successful, implement fully; if not, restart cycle

Benchmarking

Benchmarking is the process of comparing an organization's processes and performance metrics to industry best practices or competitors. It identifies performance gaps and areas for improvement.

Types of Benchmarking

- Internal Benchmarking — Comparing departments within the same organization
- Competitive Benchmarking — Comparing with direct competitors

- Functional Benchmarking — Comparing similar functions across different industries
- Generic Benchmarking — Comparing best practices regardless of industry

> ✅ **EXAM TIP: Write the 4-phase PDCA diagram in words. Examiners like when you show the cyclic nature — it's not a one-time process.**

Q5. Statistical Process Control (SPC) & Cultural Change in Quality

Statistical Process Control (SPC)

SPC is a method of quality control using statistical methods to monitor and control a process. It helps ensure processes operate at their full potential to produce conforming products.

Key Tools of SPC

- Control Charts — Track process variation over time (most important tool)
- Histograms — Show distribution of process data
- Pareto Charts — Identify the most frequent defect causes (80/20 rule)
- Cause-and-Effect Diagram (Fishbone/Ishikawa) — Identify root causes of defects
- Scatter Diagrams — Show relationship between two variables

Cultural Change in Quality Management

For quality to succeed, an organization must change its culture. This means: shifting from 'quality is the QA team's job' to 'quality is everyone's responsibility'. Key aspects: management commitment, training employees, rewarding quality behavior, and open communication about problems.

MODULE 2 — Software Testing Fundamentals

STQA Exam Prep | Top Questions with Full Answers | Based on Past Paper Analysis

Q1. Software Testing Lifecycle (STLC) & Testing Principles

★ **Appeared in BOTH past papers — guaranteed**

Software Testing Life Cycle (STLC)

Phase	Key Activities	Output
Requirement Analysis	Understand & analyze test requirements	RTM, Feasibility Report
Test Planning	Define scope, strategy, resources, schedule	Test Plan Document
Test Case Design	Write test cases, test data preparation	Test Cases, Test Data
Environment Setup	Configure hardware/software for testing	Test Environment Ready
Test Execution	Run test cases, log defects	Test Execution Report
Test Closure	Evaluate completion criteria, prepare final report	Test Closure Report

7 Principles of Software Testing

- Testing shows presence of bugs — not absence
- Exhaustive testing is impossible — use risk-based approach
- Early testing saves time and cost
- Defect clustering — most bugs found in small number of modules
- Pesticide paradox — same tests repeated become ineffective; update test cases
- Testing is context dependent — different apps need different testing
- Absence of error fallacy — software with no bugs but wrong functionality is still a failure

> ✅ **EXAM TIP: These 7 principles are easy marks. Memorize them as a numbered list and write one sentence explaining each.**

Q2. Defects — Definition, Categories & Requirement Traceability Matrix

★ **Asked in both papers — core topic**

What is a Defect?

A defect (bug/fault) is any deviation of the software from its expected behavior as defined in requirements. It occurs when the actual output does not match the expected output.

Categories of Defects

Category	Description	Example
Wrong	Feature implemented incorrectly	Login shows wrong error message
Missing	Requirement not implemented	Export button absent
Extra	Feature added not in requirements	Extra unrequested field on form

Performance	Software too slow or uses too much memory	Page loads in 10s instead of 2s
Security	Vulnerability in software	Password stored in plain text

Requirement Traceability Matrix (RTM)

RTM is a document that maps requirements to test cases. It ensures all requirements are tested and provides traceability from requirement to test execution.

Req ID	Requirement	Test Case ID	Status
R001	User Login	TC001, TC002	Pass
R002	Password Reset	TC003	Fail
R003	Dashboard View	TC004, TC005	Pass

Q3. V-Model of Testing — Advantages & Disadvantages

★ Asked in both past papers — draw the diagram!

V-Model Definition

The V-Model is a software development and testing model where each development phase has a corresponding testing phase. It is shaped like a letter V — development on the left, testing on the right.

V-Model Phases

Development (Left Side)	Corresponding Test Phase (Right Side)
Requirements Analysis	Acceptance Testing
System Design	System Testing
Architecture Design	Integration Testing
Module Design	Unit Testing
Coding (Bottom of V)	

Advantages of V-Model

- Each development stage has a corresponding test stage — errors caught early
- Simple and easy to manage — clear milestones
- Works well for small to medium-sized projects with stable requirements
- Testing is planned proactively, not reactively

Disadvantages of V-Model

- Rigid and inflexible — poor for changing requirements
- No working software produced early in the cycle
- Risk is not handled well
- Not suitable for complex, object-oriented, or agile projects

> **✓ EXAM TIP: Draw the V-shape diagram even in a written exam. Left side: Requirements → Design → Code. Right side: Unit Test → Integration Test → System Test → Acceptance Test.**

Q4. Verification & Validation — Difference & Activities

★ High chance — asked in both papers

Key Difference

Aspect	Verification	Validation
Definition	Are we building the product right?	Are we building the right product?
Focus	Process-oriented	Product-oriented
Activities	Reviews, walkthroughs, inspections	Testing (unit, integration, system)
Done by	QA team, developers	Testing team, users
When	During development phases	After development (or end of phase)
Goal	Conforms to specs and standards	Meets user needs and expectations

> ✅ **EXAM TIP: Remember: Verification = 'V for building right'. Validation = 'V for right product'. Easy mnemonic!**

Q5. Test Strategy, Maintenance Testing & Testing Skills

Test Strategy

A test strategy is a high-level document that outlines the testing approach for a project. It includes: scope, test levels, test types, entry/exit criteria, risk assessment, and tools to be used.

Stages in Developing Test Strategy

- Define the scope and objectives
- Identify testing types needed (functional, performance, security)
- Plan resources, timelines, and environment
- Identify risks and mitigation strategies
- Define reporting and defect management process

Maintenance Testing

Maintenance testing is testing performed after changes are made to a system in production. Types: Corrective (fixing defects), Adaptive (changes due to environment), Perfective (improvements), Preventive (future-proofing).

Skills Required by a Tester

Skill Category	Examples
Technical Skills	Understanding of programming, databases, OS, testing tools
Analytical Skills	Ability to analyze requirements and identify test scenarios
Communication	Clearly document and report defects, coordinate with developers
Attention to Detail	Find subtle bugs others may miss
Domain Knowledge	Understanding of the business domain being tested

MODULE 3 — Test Case Design Techniques

STQA Exam Prep | Top Questions with Full Answers | Based on Past Paper Analysis

Q1. Equivalence Class Partitioning — Types & Examples

★ **Asked in BOTH past papers — very important**

Definition

Equivalence Class Partitioning (ECP) is a black-box testing technique that divides input data into partitions (equivalence classes) where all values in a class are expected to behave the same way. If one value in a class works, all should work; if one fails, all should fail.

Types of Equivalence Classes

Type	Description	Example (Age field: 18–60)
Valid Equivalence Class	Values the system should accept	35 (valid)
Invalid Equivalence Class	Values the system should reject	-5 (too low), 100 (too high)

Example — Login Field (Username: 6–12 characters)

Class	Range	Test Value	Expected Result
Valid	6–12 chars	"testuser"	Accept
Invalid (too short)	< 6 chars	"abc"	Reject
Invalid (too long)	> 12 chars	"abcdefghijklmn"	Reject

Types: Weak vs Strong ECP

- Weak ECP — Test one value per equivalence class (minimum test cases)
- Strong ECP — Test one value per combination of equivalence classes (more thorough)

> ✅ **EXAM TIP: Always draw the number line showing the valid and invalid partitions. It earns extra marks visually.**

Q2. Boundary Value Analysis (BVA) — Normal & Robust

★ **Asked in both past papers — guaranteed**

Definition

Boundary Value Analysis focuses on testing at the boundaries of equivalence classes because defects tend to cluster at boundaries. For each boundary, we test: just below, at, and just above the boundary.

Normal BVA (for input range: min to max)

Test values: min-1, min, min+1, nominal, max-1, max, max+1

Example — Age input valid range: 18 to 60

Test Value	Type	Expected Result
17	min - 1 (just below min)	Invalid
18	min (boundary)	Valid

19	min + 1	Valid
39	Nominal (middle)	Valid
59	max - 1	Valid
60	max (boundary)	Valid
61	max + 1 (just above max)	Invalid

Robust BVA

Robust BVA extends Normal BVA by also testing values below min-1 and above max+1 (i.e., min-2 and max+2), making it more comprehensive for detecting robustness-related defects.

Q3. Cause-and-Effect Graphing & Decision Table Testing

★ **High chance — both papers**

Cause-and-Effect Graphing

Cause-and-Effect graphing is a systematic technique for identifying combinations of inputs (causes) that produce specific outputs (effects). It visually represents logical relationships between conditions.

Steps in Cause-and-Effect Graphing

- Identify causes (input conditions) and effects (output actions)
- Build the cause-effect graph showing logical relationships
- Add constraints (E = exclusive, I = inclusive, O = one and only one, R = requires)
- Convert the graph into a decision table
- Derive test cases from the decision table

Decision Table Testing

A decision table lists all possible combinations of conditions and their corresponding actions. It ensures all condition combinations are tested.

Conditions/Actions	Rule 1	Rule 2	Rule 3	Rule 4
Condition A (T/F)	T	T	F	F
Condition B (T/F)	T	F	T	F
Action X	✓	✗	✗	✗
Action Y	✗	✓	✓	✗

Q4. DD Path Testing & Basis Path Testing (McCabe's Method)

★ **High chance — asked in both papers**

DD Path Testing

A DD path (Decision-to-Decision path) is a path through a program's control flow graph between two decision points. DD path testing ensures all paths between decisions are executed.

McCabe's Cyclomatic Complexity (Basis Path Testing)

Cyclomatic Complexity (CC) measures the number of linearly independent paths in a program. It equals the minimum number of test cases needed to cover all paths.

Formula for Cyclomatic Complexity

Formula	Description
CC = E - N + 2P	E = edges, N = nodes, P = connected components (usually 1)
CC = D + 1	D = number of decision nodes (if, while, for, etc.)
CC = Number of regions	In a planar flow graph, regions include the outer region

McCabe's Baseline Method

- Step 1: Draw the control flow graph
- Step 2: Calculate cyclomatic complexity (CC)
- Step 3: Find baseline path (1 complete path through the graph)
- Step 4: Flip each decision to generate CC-1 more paths
- Step 5: Each path = one test case

> **✅ EXAM TIP: Draw a simple flowchart example (e.g., if-else with a loop) and show how CC = 3 → 3 test cases needed.**

Q5. Program Slicing, Data Flow Testing & Define-Use Testing

Program Slicing

A program slice is the set of program statements that affect the value of a variable at a specific point. It is used to isolate the relevant code for debugging and testing.

Types of Slices

- Static Slice — All statements that may affect a variable's value (computed without running the program)
- Dynamic Slice — Only statements that did affect the variable during a specific execution

Data Flow Testing

Data flow testing focuses on the points where variables receive values (definitions) and where those values are used. It ensures all define-use pairs are tested.

Define-Use (DU) Testing

Concept	Description
Definition (DEF)	A statement where a variable is assigned a value
Use (USE)	A statement where a variable's value is referenced
P-Use	Variable used in a predicate (condition) — affects branching
C-Use	Variable used in a computation
DU Path	A path from a definition to a use without another definition in between

MODULE 4 — Verification, Validation & V-Model

STQA Exam Prep | Top Questions with Full Answers | Based on Past Paper Analysis

Q1. Verification Workbench — Structure & Entities

★ **Asked in BOTH past papers**

What is a Verification Workbench?

A verification workbench is a structured environment that supports static verification activities like reviews, walkthroughs, and inspections. It is the 'place' where verification is conducted.

Entities Involved in Verification Workbench

Entity	Role
Author/Producer	Person who created the work product to be verified
Moderator	Leads the review process, ensures it stays on track
Reviewer/Inspector	Checks the work product for defects against standards
Scribe/Recorder	Documents all issues, defects, and decisions found
Management	Provides resources, monitors process, receives reports

Verification vs Validation Workbench

- Verification Workbench = static activities (reviews, inspections) — checks the process
- Validation Workbench = dynamic activities (actual testing) — checks the product

Q2. Types of Reviews (Based on Stage/Phase)

★ **Asked in both past papers — must know**

4 Main Types of Reviews

Review Type	Description	Formality	When Used
Informal Review	Casual review by a colleague, no formal process	Low	Any phase, quick checks
Walkthrough	Author explains work to peers, who ask questions	Medium	Requirements, design phase
Technical Review	Technical evaluation by qualified team members	Medium-High	Design, code phase
Inspection (Fagan)	Highly formal, structured, roles defined (moderator, inspector, recorder)	Very High	Any formal deliverable

Fagan Inspection — 6 Steps

- Planning — Entry criteria checked, team assembled, materials distributed
- Overview — Author presents the document
- Preparation — Each inspector studies the document individually
- Inspection Meeting — Team discusses and logs all defects

- Rework — Author fixes defects
- Follow-up — Moderator verifies all defects are resolved

> ✅ **EXAM TIP: Fagan Inspection always comes in the exam. Write all 6 steps and state that it catches 60-90% of defects.**

Q3. Acceptance Testing & Its Types

★ High chance — both papers

Definition

Acceptance testing is formal testing conducted to determine whether a system meets business requirements and whether it should be accepted by the customer or user.

Types of Acceptance Testing

Type	Description	Conducted By
User Acceptance Testing (UAT)	End users test the system in a real-world scenario	Business users
Operational Acceptance Testing (OAT)	Tests operational aspects: backup, recovery, security	IT operations
Alpha Testing	Testing at developer's site by a selected group of customers	Selected customers
Beta Testing	Testing at customer's site with real users before full release	Real users in production

Q4. VV Model — Roles, Responsibilities & Management of V&V

VV Model (Verification and Validation Model)

The VV Model extends the V-Model by explicitly incorporating both verification (process-oriented) and validation (product-oriented) at every phase of the development lifecycle.

Key Roles in VV Model

Role	Responsibility
V&V Leader	Plans and coordinates all verification and validation activities
Software Tester	Executes test cases and logs defects
Software Developer	Corrects defects found during verification and validation
Quality Assurance	Ensures V&V processes follow standards and produce quality outputs
Customer	Validates that final system meets their requirements

Management of V&V

- Planning — Define V&V tasks, resources, schedule for each phase
- Monitoring — Track progress against the V&V plan
- Reporting — Regular reports on V&V status to management and customer
- Closure — Verify all V&V tasks completed and exit criteria met

Q5. Audit — Definition & Types

What is an Audit?

An audit is an independent, systematic examination to determine whether activities and related results comply with planned arrangements and whether these are implemented effectively.

Types of Audits

Audit Type	Description
Process Audit	Checks whether development processes follow defined standards
Product Audit	Evaluates a product against its requirements/specifications
Project Audit	Reviews project plan, schedule, and resource usage
Compliance Audit	Checks compliance with laws, regulations, or organizational policies
Internal Audit	Conducted by the organization's own team
External Audit	Conducted by an independent third party (e.g., ISO audit)

MODULE 5 — Testing Levels, Integration & Tools

STQA Exam Prep | Top Questions with Full Answers | Based on Past Paper Analysis

Q1. Levels of Testing — Unit, Integration, System, Acceptance

★ **Asked in both past papers — must know**

4 Levels of Testing

Level	What is Tested?	Done By	Technique
Unit Testing	Individual functions/modules in isolation	Developers	White-box testing
Integration Testing	Interaction between combined modules	Dev/Test team	Black/White box
System Testing	Complete integrated system vs requirements	Test team	Black-box testing
Acceptance Testing	Full system vs business requirements	Users/Customers	Scenario-based

Q2. Integration Testing — Types & Big Bang Approach

★ **High chance — both papers**

What is Integration Testing?

Integration testing tests the interfaces and interactions between combined software modules. It ensures modules work correctly when combined together.

Types of Integration Testing

Type	Description	Advantage	Disadvantage
Big Bang	All modules integrated at once and tested together	Simple — just put it all together	Hard to isolate defects; no early testing
Top-Down	Start from top-level modules, use stubs for lower	Tests main flow early	Lower modules tested late
Bottom-Up	Start from lower modules, use drivers for upper	Tests real modules early	Top-level logic tested late
Sandwich (Hybrid)	Combines top-down and bottom-up	Balanced approach	Complex to manage

Sandwich Testing in Detail

Sandwich (hybrid) testing divides the system into three layers: top, middle (target), and bottom. Top layer uses top-down approach with stubs; bottom layer uses bottom-up approach with drivers. Both proceed simultaneously toward the middle layer.

Advantages of Sandwich Testing

- Both top-level and bottom-level modules are tested early
- Reduces need for stubs and drivers compared to pure top-down or bottom-up

- Suitable for large complex systems

Q3. Static & Dynamic Testing Tools

★ **High chance — both papers**

Static Testing Tools

Static testing tools analyze code without executing it. They find defects early, often during code review or inspection.

Tool Type	Purpose	Examples
Code Analyzers	Check code for syntax/style errors	ESLint, PMD, Checkstyle
Complexity Analyzers	Measure cyclomatic complexity	McCabe Tool, SonarQube
Review Support Tools	Manage inspection checklists and defect logs	Crucible, Gerrit
Compiler Warnings	Catch issues at compile time	GCC, Javac

Dynamic Testing Tools

Dynamic testing tools execute the software and analyze runtime behavior.

Tool Type	Purpose	Examples
Functional Testing	Automate test case execution	Selenium, UFT
Performance Testing	Test speed, load, stress	JMeter, LoadRunner
Regression Testing	Rerun tests after changes	Selenium, TestNG
Code Coverage	Measure % of code tested	JaCoCo, Istanbul

> ✅ **EXAM TIP: Know at least 2 examples of each tool type. Examiners love tool names as specific answers.**

Q4. Automated Testing — Benefits, Challenges & Tool Selection

★ **High chance — past papers**

Benefits of Automation

- Faster execution — automated tests run much faster than manual tests
- Reusable — write once, run many times (especially valuable for regression testing)
- Consistent — no human error in execution
- 24/7 execution — tests can run overnight without human presence
- Better coverage — more scenarios tested in less time

Challenges of Introducing Automation

- High initial investment — tools are expensive and setup takes time
- Maintenance overhead — test scripts need updating when UI changes
- Not all tests can be automated — exploratory, usability testing needs humans
- Skill requirement — testers need programming knowledge
- False sense of security — automation tests only what they are programmed for

Tool Selection Criteria

Criterion	Consideration

Technology Support	Does it support the application's tech stack (Java, .NET, web)?
Ease of Use	Learning curve for the team
Cost	Licensing fees vs open-source options
Vendor Support	Community, documentation, updates
Integration	Works with CI/CD pipeline, bug tracker, etc.

Q5. Performance Testing, Security Testing & Requirement-Based Testing

Performance Testing

Performance testing evaluates the speed, scalability, and stability of a system under load.

Type	What it Tests
Load Testing	System behavior under expected load (normal usage)
Stress Testing	System behavior beyond normal capacity (breaking point)
Spike Testing	System behavior under sudden surge in load
Endurance Testing	System behavior under sustained load over time (memory leaks)
Volume Testing	System behavior with large amounts of data

Security Testing

Security testing verifies that the software protects data and functionality from unauthorized access. Key areas: authentication, authorization, data encryption, input validation (SQL injection, XSS), and session management.

Requirement-Based Testing — Stages

- Proposal Testing — Test against the initial project proposal
- Requirement Testing — Verify each functional requirement has test coverage
- Design Testing — Verify design meets requirements
- Code Testing — Unit and integration testing of implemented code
- Acceptance Testing — Final validation against business requirements

> **✅ EXAM TIP: For performance testing, always name at least 3 types. 'Performance testing' alone is not enough — specify load, stress, spike etc.**

QUICK CHEAT SHEET — KEY TERMS

Term	One-Line Definition
Quality	Fitness for purpose; meeting stated and implied requirements
TQM	Management philosophy of continuous quality improvement by all employees
PDCA Cycle	Plan-Do-Check-Act: cyclic continuous improvement model
QMS	Formalized system documenting processes to achieve quality (e.g., ISO 9001)
Benchmarking	Comparing performance against industry best practices
SPC	Using statistics to monitor and control process quality
V-Model	Development-testing model where each dev phase has a test phase
Verification	Building the product right (process-oriented, static)
Validation	Building the right product (product-oriented, dynamic)
ECP	Dividing inputs into classes where all values behave the same
BVA	Testing at and near the boundaries of input partitions
Cyclomatic Complexity	Number of linearly independent paths (CC = E - N + 2P)
DD Path	Path between two decision nodes in a control flow graph
RTM	Matrix mapping requirements to test cases for traceability
UAT	End-user testing to verify system meets business needs
Big Bang Integration	All modules combined and tested at once
Sandwich Testing	Hybrid integration testing — top-down + bottom-up simultaneously
Static Tools	Analyze code without execution (e.g., ESLint, SonarQube)
Dynamic Tools	Test code by executing it (e.g., Selenium, JMeter)
Defect	Any deviation from expected behavior as defined in requirements